

Buffer Overflow Security Project — Comprehensive Guide

Course Project | Security & Penetration Testing

Team Size: Up to 5 Members

Table of Contents

1. Project Overview
 2. Project Architecture & File Map
 3. Environment Setup (Docker)
 4. Part A — Buffer Overflow Theory
 5. Part B — The Buffer Lab (Python Stack Simulator)
 6. Part C — Modern Pentest Lab (64-bit C Exploitation)
 7. Part D — Mitigations (5 Defense Techniques)
 8. Part E — Bypass: Defeating Protections (ROP & Leaks)
 9. Part F — Bonus: Buffer Overflow Across Software Types
 10. Part G — Testing & Validation
 11. Part H — Team Roles & Timeline
 12. References & Further Reading
-

1. Project Overview

This project demonstrates a complete, modern security workflow around **Stack-Based Buffer Overflow** vulnerabilities. It moves beyond outdated 32-bit Windows tutorials to teach real-world 64-bit Linux exploitation using industry-standard tooling.

- **Attack:** Exploit a vulnerable C application across three progressive difficulty levels to gain code execution.
- **Defend:** Apply defense-in-depth mitigations (Canaries, NX, PIE, bounded copies) to prevent the attack.
- **Bypass:** Show how advanced techniques like Return-Oriented Programming (ROP) and Information Leaks can defeat modern compiler protections.
- **Analyze:** Study how buffer overflows affect different software categories (Web, Mobile, Desktop, IoT).

The project is fully Docker-based and includes complementary lab environments:

Lab	Technology	Purpose
buffer_lab	Python CLI	Safe stack-frame simulator for understanding basic memory layout
college-pentest-lab	Docker + C/Python	Real-world 64-bit Linux exploitation with pwntools and GDB
interactive_lab	Browser HTML/JS	Visual byte-level stack simulation

2. Project Architecture & File Map

```

buffer/
├─ buffer_lab/                # Python stack simulator (Theory)
├─ examples/                  # Interactive browser visualization
├─ college-pentest-lab/      # Main Docker Pentest Environment
│   ├── docker-compose.yml    # 5 services: attacker, safe, levels 1-3
│   ├── README.md             # Lab overview
│   ├── START_HERE.md         # Quick-start guide
│   └─ attacker/
│       ├── Dockerfile         # Kali-style image with pwntools, GDB, ROPgadget
│       └─ scripts/           # Exploit scripts
│           ├── 00_recon.py     # Server banner grabbing & checksec
│           ├── 01_fuzzer.py    # Fuzzes TRUN command to find crash
│           ├── 02_find_offset.py # Cyclic pattern to find RIP offset
│           ├── 03_exploit_level1.py # ret2shellcode (No protections)
│           ├── 04_exploit_level2.py # ROP / ret2libc (NX bypass)
│           ├── 05_exploit_level3.py # Format String Leak + ROP (Full bypass)
│           └─ 06_verify_safe.py # Validates safe_server blocks attacks
│       └─ victim/
│           ├── Dockerfile     # Multi-stage C compilation
│           ├── vuln_server.c   # Vulnerable TCP Server (strcpy & printf)
│           ├── safe_server.c   # Remediated Server (strncpy & input validation)
│           └─ flag.txt         # Target file
├─ tools/
│   └─ build_guide_pdf.py      # Markdown-to-PDF builder
└─ docs/
    └─ COMPREHENSIVE_PROJECT_GUIDE.md # This document

```

3. Environment Setup (Docker)

Prerequisites

- Docker Desktop (Windows/Mac) or Docker Engine (Linux)
- Docker Compose v2+
- A terminal (PowerShell, bash, or zsh)

Starting the Docker Lab

```
cd college-pentest-lab
docker compose up --build -d
```

This launches five interconnected containers on an isolated `labnet` bridge network:

Container	Port	Architecture	Description
attacker	—	x86_64	Exploit dev environment (pwntools, GDB, pwndbg)
victim-11	9999	x86_64	Level 1: No protections (ret2shellcode)
victim-12	9998	x86_64	Level 2: NX Enabled (ROP/ret2libc)
victim-13	9997	x86_64	Level 3: NX + Canary + PIE (Leak + ROP)
safe-victim	9996	x86_64	Safe: All protections + safe code

Accessing the Exploit Environment

All attacks should be executed from within the attacker container, which comes pre-configured with necessary tooling:

```
docker exec -it attacker bash
```

4. Part A — Buffer Overflow Theory

What Is a Buffer Overflow?

A **buffer overflow** occurs when a program writes data beyond the boundary of a fixed-size memory buffer, corrupting adjacent memory space.

The Stack Layout (x86_64)

The stack stores critical control data alongside local variables. On a 64-bit architecture, pointers are 8 bytes long.

```

+-----+ Higher Addresses
|   Function Arguments   |
+-----+
|   Return Address (RIP) | <-- TARGET: 8 bytes. Controls execution flow
+-----+
|   Saved RBP            | <-- 8 bytes. Base pointer of caller
+-----+
|   [Stack Canary]      | <-- Optional: 8 byte corruption detector
+-----+
|   Local Buffer         | <-- 128 bytes. Our input goes here
+-----+ Lower Addresses (stack grows down)

```

When an attacker writes more than 128 bytes into the local buffer, they overwrite the saved RBP and the Return Address (RIP). When the function executes the `ret` instruction, the CPU pops the attacker's value off the stack into the instruction pointer, hijacking the program.

The Dangerous C Functions

Dangerous	Safe Alternative	Problem
<code>strcpy()</code>	<code>strncpy()</code>	Copies until <code>\0</code> with no length check
<code>gets()</code>	<code>fgets()</code>	Reads until newline with no length check
<code>sprintf()</code>	<code>snprintf()</code>	Formats string with no length check
<code>printf(input)</code>	<code>printf("%s", input)</code>	Format String Vulnerability

5. Part B — The Buffer Lab (Python Stack Simulator)

Before jumping into GDB and x86_64 assembly, students can use the `buffer_lab` package. It is a safe, pure-Python simulator that models stack memory.

```

# Generate a cyclic pattern to find offsets
python -m buffer_lab pattern --length 150

# Test how a canary works
python -m buffer_lab demo-overflow --buffer-size 64 --payload-size 80 --canary
# Output: ABORTED: stack canary detected corruption

```

6. Part C — Modern Pentest Lab (64-bit C Exploitation)

The core of this project is a custom TCP server (`vuIn_server.c`) written in C. It exposes two commands:

- `TRUN <data>`: Vulnerable to a classic `strcpy` buffer overflow.
- `INFO <data>`: Vulnerable to a `printf` format string information leak (Level 3 only).

Exploitation Workflow

All scripts are located in `attacker/scripts/`.

Step 1: Reconnaissance

The attacker maps the network and checks binary protections using `checksec`.

```
python3 scripts/00_recon.py
```

Step 2: Fuzzing

The attacker sends increasingly large payloads to `TRUN` until the server stops responding, identifying the approximate size of the buffer.

```
python3 scripts/01_fuzzer.py victim-l1 9999
```

Step 3: Finding the Offset

Using a de Bruijn sequence (cyclic pattern), the attacker determines the exact byte offset required to overwrite the Return Address (RIP). In our lab, the buffer is 128 bytes + 8 bytes (RBP) = **136 bytes to RIP**.

```
python3 scripts/02_find_offset.py victim-l1 9999
```

Step 4: Level 1 Exploit (ret2shellcode)

Level 1 is compiled with **no protections** (`-fno-stack-protector -z execstack -no-pie`).

- **Technique:** Inject x86_64 `execve("/bin/sh")` shellcode directly onto the stack.
- **Execution:** Overwrite RIP with the stack's address to jump to the shellcode.

```
python3 scripts/03_exploit_level1.py
```

7. Part D — Mitigations (5 Defense Techniques)

To secure the server, `safe_server.c` implements defense-in-depth, combining software-level fixes with compiler protections.

1. Code-Level Fixes

- Replaced `strcpy` with bounded `strncpy`.
- Enforced strict length validation **before** copying data.

- Fixed the format string vulnerability by forcing `snprintf(buf, size, "%s", input)`.

2. Compiler Protections

Protection	Compile Flag	How it Works
NX / DEP	<code>-z noexecstack</code>	Marks the stack as non-executable. Stops <code>ret2shellcode</code> .
Stack Canary	<code>-fstack-protector-all</code>	Places a random 8-byte value before the return address. Checked upon <code>ret</code> .
PIE / ASLR	<code>-pie -fPIE</code>	Randomizes the base addresses of the binary and libraries in memory.
Full RELRO	<code>-Wl,-z,relro,-z,now</code>	Makes the Global Offset Table (GOT) read-only, preventing function hook hijacks.
FORTIFY_SOURCE	<code>-D_FORTIFY_SOURCE=2</code>	Replaces unsafe standard library calls with bounded versions at compile time.

Verification:

```
python3 scripts/06_verify_safe.py
# Result: All exploitation attempts are blocked.
```

8. Part E — Bypass: Defeating Protections (ROP & Leaks)

Modern exploitation rarely relies on executing shellcode on the stack due to NX/DEP. The project demonstrates how advanced techniques bypass these mitigations.

Level 2: Bypassing NX via ROP (Return-Oriented Programming)

Level 2 enables NX, meaning the stack is no longer executable.

- The Bypass:** Instead of injecting code, we string together existing snippets of executable code in the binary (called "gadgets").
- Technique:** We find a `pop rdi; ret` gadget, load the address of the string `"/bin/sh"` into the `RDI` register, and return into the `system()` function located in `libc`.

```
python3 scripts/04_exploit_level2.py
```

Level 3: Bypassing Canaries and PIE via Info Leaks

Level 3 enables Canaries and PIE. We cannot guess the Canary, nor do we know where the ROP gadgets are located in memory.

- **The Bypass:** We exploit a separate vulnerability (Format String in the `INFO` command) to leak memory contents before we overflow the buffer.
- **Technique:**
 1. Send `%p.%p.%p...` to read values off the stack.
 2. Parse the leaked Stack Canary to place it correctly in our overflow payload, keeping the server from aborting.
 3. Parse leaked addresses to dynamically calculate the randomized PIE and Libc base addresses.
 4. Construct the ROP chain dynamically on the fly.

```
python3 scripts/05_exploit_level3.py
```

9. Part F — Bonus: Buffer Overflow Across Software Types

Web Applications

How it affects web:

- Web servers (Apache, Nginx) and frameworks often rely on underlying C/C++ libraries (e.g., OpenSSL, libxml2, image parsers).
- **Example:** Heartbleed (CVE-2014-0160) was a buffer over-read in OpenSSL that allowed attackers to leak memory, exposing passwords and private keys from web servers.

Mobile Applications

How it affects mobile:

- Android apps are mostly written in memory-safe Java/Kotlin, but heavily rely on the **Java Native Interface (JNI)** for performance. Vulnerabilities in these C/C++ JNI libraries can lead to remote code execution on the device.
- iOS apps utilizing legacy Objective-C or C libraries face similar risks.

Desktop Applications

How it affects desktop:

- Document parsers (PDF readers, Office suites) and media players are historically prime targets.
- **Example:** Opening a malformed PDF file triggers an overflow in Adobe Reader's parsing engine, executing the embedded payload and compromising the user's PC simply by opening a document.

IoT / Embedded Systems

How it affects IoT:

- IoT devices (routers, IP cameras) run on limited hardware with minimal OS protections (often lacking ASLR, DEP, and Canaries).
- Their firmware is written in C and rarely updated, making them the most vulnerable software category today.
- **Example:** The Mirai Botnet compromised millions of IoT devices via simple buffer overflows and default credentials to launch massive DDoS attacks.

10. Part G — Testing & Validation

The lab is validated directly via exploitation. The success criteria for the project is obtaining arbitrary command execution (a shell) on the vulnerable containers, and proving the safe container cannot be breached.

1. **Build the environment:** `docker compose up --build -d`
2. **Access attacker:** `docker exec -it attacker bash`
3. **Validate exploitability:** Run `python3 scripts/03_exploit_level1.py`. If a shell interaction loop starts, the vulnerability is active.
4. **Validate mitigation:** Run `python3 scripts/06_verify_safe.py`. The script will test all attack vectors (fuzzing, overflowing, format string injection) and must report them all as `BLOCKED`.

11. Part H — Team Roles & Timeline

Recommended Team Distribution (5 Members)

Role	Member	Responsibilities	Deliverables
Project Lead	(Name)	Coordination, Docker networking, final documentation review	Architecture diagram, <code>README.md</code>
Exploit Dev (Theory)	(Name)	Levels 1 & 2 exploits (Fuzzing, Offset calculation, ROP chains)	<code>01_fuzzer.py</code> , <code>04_exploit_level2.py</code>
Exploit Dev (Advanced)	(Name)	Level 3 exploits (Format string leaks, bypassing Canaries/ASLR)	<code>05_exploit_level3.py</code>
Defense Engineer	(Name)	Writing <code>safe_server.c</code> , implementing compiler flags	<code>safe_server.c</code> , <code>06_verify_safe.py</code>
Security Analyst	(Name)	Part H (Software types), research, presentation slides	Presentation, Part F documentation

Proposed Timeline (4 Weeks)

- **Week 1:** Environment setup, Docker configuration, Theory review (Parts A & B).
 - **Week 2:** Implementing `vuln_server.c`, fuzzing, and achieving the Level 1 `ret2shellcode` exploit.
 - **Week 3:** Advanced exploitation (ROP chains, Info leaks) for Levels 2 and 3.
 - **Week 4:** Securing the application (`safe_server.c`), writing final reports, preparing the presentation.
-

12. References & Further Reading

- [Smashing The Stack For Fun And Profit \(Aleph One\)](#) - The foundational paper on buffer overflows.
- [ROPgadget Tool](#)
- [Pwntools Documentation](#)
- [Pwndbg Features](#)
- [CWE-119: Improper Restriction of Operations within the Bounds of a Memory Buffer](#)